BayesianEstimationTool (BET) Cheat Sheet

Purpose: `BayesianEstimationTool` (BET) organizes hierarchical Bayesian estimation for datasets containing multiple related entries. Models may contain global parameters shared across entries and standardized local parameters that vary by entry. `Train` infers the population/global posterior, training locals, and local-parameter correlation. `Test` carries the learned population state to new entries and can either sample locals from the trained population or infer them from new observations with an estimator. `ScoreTrain` and `ScoreTest` evaluate predictions. `Save` and `Load` preserve the learned state and the settings needed to reconstruct how estimation was performed. Inference is supplied through an Estimator interface; built-in estimators include NUTS and SVI.
Dependencies: jax, numpyro, numpy
Installation: Install in editable mode with `python -m pip install -e "/path/to/BayesianEstimationTool"` then import with `from BayesianEstimationTool import BayesianEstimationTool, NutsEstimator, SviEstimator`. Editable installation means repository code changes are used without reinstalling. For repository examples, the parent folder may instead be added to `sys.path`.

Conventions

`arg{}` - dictionary whose values have no entry dimension; used for one entry's data, local parameters, model output, globals, or other named values.
`arg{[]}` - dictionary containing arrays spanning multiple entries or posterior samples.
`arg[]` - 1D sequence.
`Fn(...) -> value` - function argument or method together with the value it returns.
Italic arguments - accept `None` as a valid input.

Main Functions

BayesianEstimationTool(modelDataFull{[]}, ModelFn(globalParams{}, localParams{}, data{}) -> modelOut{}, *localParamNames[]*, *GlobalParamFn(dataShapes{}) -> globalParams{}*, choleskyConcentration=2.0, *rngSeed*) -> BayesianEstimationTool: Creates BET. **localParamNames** defines standardized entry-level parameters. With multiple locals, BET learns their correlation with an LKJ Cholesky prior controlled by **choleskyConcentration**. **GlobalParamFn** defines NumPyro global parameters and receives the per-entry shapes of the full data. **rngSeed** initializes BET's persistent RNG.

Train(trainIndices[], TrainLikelihoodFn(globalParams{}, localParams{[]}, data{[]}, modelOut{[]}), Estimator, printSummary=False, *rngSeed*, saveLocals=True, saveModelOut=False) -> estimatorOutput: Fits **trainIndices** with the supplied estimator. BET stores global posterior samples and the learned Cholesky state; **saveLocals** stores transformed training-local samples and **saveModelOut** stores model-output samples. **printSummary** uses BET's generic posterior summary unless the estimator replaces or augments it.

Test(testIndices[], *TestLikelihoodFn(globalParams{}, localParams{[]}, data{[]}, modelOut{[]})*, *Estimator*, *nGlobalSamples*, nLocalSamples=2000, printSummary=False, *rngSeed*, saveModelOut=False) -> None | estimatorOutput[]: Tests on new entries. **nGlobalSamples=None** uses the median trained global/Cholesky state; a positive integer samples that many trained states. With **TestLikelihoodFn=None**, no estimator is needed and BET draws **nLocalSamples** locals from the trained population. With a likelihood, **Estimator** infers test locals separately for each selected global state.

ScoreTrain(ScoreFn(modelOut{}, data{}) -> score{}, *nSamples*, *rngSeed*) -> scores{[]}: Scores training predictions. **nSamples=None** uses all available joint training-posterior samples; otherwise BET selects that many. Saved model outputs are reused when available, otherwise they are regenerated from saved parameters.

ScoreTest(ScoreFn(modelOut{}, data{}) -> score{}, *nSamples*, *rngSeed*) -> scores{[]}: Scores the latest test state. **nSamples=None** uses all stored local samples; otherwise that many are selected for each stored global state. Saved model outputs are reused when available, otherwise they are regenerated.

Save(*fileName*, saveTrainLocals=True, saveTestLocals=True, saveTrainModelOut=False, saveTestModelOut=False, saveAllData=False) -> None | record{}: Writes compressed `.npz` state, or returns a nested record when **fileName=None**. Saved metadata includes estimator class names and constructor arguments so the estimation setup can be reconstructed without serializing estimator machinery. Functions are not serialized.

BayesianEstimationTool.Load(fileName, *ModelFn*, *GlobalParamFn*, *modelDataFull*) -> BayesianEstimationTool | record{}: Loads saved state. With **ModelFn=None**, returns the saved record without reconstructing BET. Otherwise model functions are supplied explicitly; **modelDataFull** is required unless the data were saved with `saveAllData=True`.

NutsEstimator(numWarmup=2000, numSamples=2000, numChains=4, acceptProb=0.95, denseMass=False, medianSamples=50) -> NutsEstimator: Configures NUTS. **medianSamples** controls `init_to_median`; **acceptProb** is the target acceptance probability. NUTS retains posterior samples from the MCMC trajectory and prints NumPyro convergence diagnostics including n_eff and r_hat.

SviEstimator(numSteps=10000, numSamples=2000, stepSize=0.01, numParticles=1, progressBar=True, medianSamples=50, multivariate=False) -> SviEstimator: Configures stochastic variational inference. Uses `init_to_median`; **numSamples** is the number of posterior samples drawn from the fitted guide for BET. By default the guide is mean-field `AutoNormal`; **multivariate=True** uses `AutoMultivariateNormal`, which can represent posterior covariance but scales less favorably for large latent spaces.